

Chapter 9. Deploying in Monitoring LLM-Based Systems

In this chapter, we explore how to deploy an agent and the infrastructure around it. There is no single “correct” architecture as the right approach depends on the use case, latency requirements, and governance constraints.

Many agent use cases are naturally batch oriented. Agents can enrich newly ingested documents with summaries or metadata, classify support tickets, and generate offline reports. In these scenarios, running the agent as part of a scheduled batch job, much like how we tested it earlier, is often sufficient, without requiring a real-time endpoint.

Another pattern is embedding the agent directly into an application codebase. While convenient at first, this can create separation-of-concerns issues by tightly coupling application logic with agent orchestration and configuration. Over time, this makes versioning, monitoring, and governance more difficult, especially when different teams own the agent and application layers.

A more robust, production-ready approach is to deploy the agent behind a Model Serving endpoint. This introduces a clear separation between the agent and the consuming application, which makes it easier to evolve the agent independently, manage versions, and support multiple clients.

In this chapter, we follow this approach and deploy the agent using a serving endpoint.

Deploying a Databricks Model Serving Endpoint

In [Chapter 4](#), we deployed several Model Serving endpoints using Databricks SDK. While we could use the same approach for agents, that would require us to do [additional setup](#), such as provisioning a service principal with access to the MLflow experiment and configuring environment variables (such as `DATABRICKS_CLIENT_ID`,

DATABRICKS_CLIENT_SECRET , and DATABRICKS_HOST) for authentication. This can conflict with the resource authentication setup we defined earlier, and it adds unnecessary complexity.

To avoid this, we'll use the [Databricks agent framework](#). It handles deployment, tracing, and authentication out of the box, so we can focus on the agent logic rather than infrastructure.

Unity AI Gateway and Guardrails

The agent framework enables the inference table as part of the AI gateway configuration, so that we can log the payload that goes through the Model Serving endpoint. At the time of writing, payload logging was the only supported configuration available for the AI agents. More advanced features like rate limiting and cost tracking are reserved for external model endpoints and provisioned throughput and pay-per-token foundation model APIs. Because these features evolve rapidly, it is always a good idea to consult the [Databricks documentation](#) for the latest updates. [Table 9-1](#) summarizes the AI gateway features that are currently supported and not supported.

Table 9-1. Supported AI gateway features by endpoint type

Feature	External model endpoint	Foundation Model API provisioned throughput endpoint	Foundation Model API pay-per-token endpoint	Custom model endpoint: <i>mlflow.pyfunc.PythonModel</i>	AI agent endpoint: <i>mlflow.pyfunc.ResponsesAgent</i>
Permission and rate limiting	Supported	Supported	Supported	Supported	<i>Not supported</i>
Payload logging	Supported	Supported	Supported	Supported	Supported
Usage tracking	Supported	Supported	Supported	Supported	<i>Not supported</i>
AI Guardrails	Supported	Supported	Supported	<i>Not supported</i>	<i>Not supported</i>
Fallbacks	Supported	<i>Not supported</i>	<i>Not supported</i>	<i>Not supported</i>	<i>Not supported</i>
Traffic splitting	Supported	Supported	<i>Not supported</i>	Supported	<i>Not supported</i>

Guardrails on the LLM level

It is worth noting that when we're deploying an AI agent, AI guardrails are not supported at the gateway level of a serving endpoint. However, we can still apply guardrails at the level of the underlying LLM used by the agent.

In our case, we use the Databricks-hosted `gpt-oss-120b` model. On top of it, we can enable guardrails for PII (blocking or redaction), jailbreak detection, hallucination detection, and unsafe content, or we can define our own custom guardrails (see [Figure 9-1](#)).¹

Create guardrail

×

Guardrail type

PII Redaction

PII Blocking

Unsafe Content

Jailbreak

Hallucination

Custom

How it works: Detects PII in the input and replaces it with placeholders before the model call. [Learn more about guardrails.](#)

Phase

Select one or both phases where this guardrail will run.

☒ **Input guardrails** Before the LLM

☐ **Output guardrails** After the LLM

Endpoint

databricks-gpt-oss-120b

Prompt

Advanced options

Mode

☒ **Enforce**

Apply the configured action (block or sanitize) when a violation is detected.

☐ **Log**

Record violations without blocking or modifying requests.

Figure 9-1. AI gateway settings for creating custom guardrails

Each guardrail can run in enforce or log mode. In *enforce mode*, the guardrail acts on a violation: it blocks the request or redacts the offending content before the response reaches the user. In *log mode*, it takes no action and only records that the violation occurred. Log mode lets you see how often a guardrail would fire against real traffic without affecting anyone, so a common pattern is to start in log mode, confirm that the guardrail behaves as expected, and switch to enforce once you trust it.

While these guardrails operate at the model level rather than the endpoint gateway, they still play an important role in ensuring safe and compliant agent behavior.

Guardrails as part of the agent logic

It's also worth noting that guardrails can (and should) be incorporated directly into the agent logic. While LLMs are inherently flexible, this flexibility can lead to undesired behavior if left unchecked. For example, in a recipe-generation app, you may want to prevent users from asking unrelated questions, such as by making Python programming queries. You can address this by adding custom validation logic, for instance, by using `mlflow.genai.evaluate()` to enforce domain constraints.

MLflow also supports *Guardrails AI scorers*. [Guardrails AI](#) provides a wide range of validators for safety, PII detection, content quality, and more. Importantly, these validators are *not always LLM-based*—many rely on rule-based approaches such as regex or keyword matching. This makes them faster, cheaper, and more predictable, which is especially useful in production systems.

MLflow includes the following built-in validators:²

ToxicLanguage

This detects whether the output contains toxic or offensive language.

NSFWText

This flags explicit or not-safe-for-work content.

DetectJailbreak

This identifies jailbreak or prompt injection attempts in the input.

DetectPII

This checks for personally identifiable information in the output.

SecretsPresent

This detects API keys, tokens, and other sensitive secrets.

GibberishText

This identifies nonsensical and low-quality text.

Here, for example, is how you can apply some of these guardrails as code:

```
import mlflow
from mlflow.genai.scorers.guardrails import ToxicLanguage, DetectPII
```

```

eval_dataset = [
    {
        "inputs": {"query": "How do I contact support?"},
        "outputs": "Let me forward you to the support team.",
    },
]

results = mlflow.genai.evaluate(
    data=eval_dataset,
    scorers=[
        ToxicLanguage(threshold=0.7),
        DetectPII(),
    ],
)

```

Implementing deployment

The *chapter9_2.deploy_agent.py* notebook demonstrates how to deploy the agent using the Databricks agent framework. We start by retrieving the model version, using the `latest-model` alias from MLflow. We then pass this version to the `agents.deploy` function, along with the endpoint configuration and the required environment variables.

These environment variables include `git_sha`, `model_version`, and `endpoint_name`, which are attached to MLflow traces for observability, to make it easy to filter and analyze runs later. In this example, `git_sha` is hard-coded, but in practice, it would be injected at deployment time via the CD pipeline. We also provide credentials for accessing Lakebase via secrets, along with the MLflow experiment ID used for storing traces:

```

from databricks import agents
from databricks.sdk import WorkspaceClient
from mlflow import MlflowClient

from arxiv_curator.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")

model_name = f"{cfg.catalog}.{cfg.schema}.arxiv_agent"
endpoint_name = "arxiv-agent-endpoint"
secret_scope = "arxiv-agent-scope"

model_version = MlflowClient().get_model_version_by_alias(
    model_name, "latest-model").version

workspace = WorkspaceClient()
experiment = MlflowClient().get_experiment_by_name(cfg.experiment_path)

```

```

git_sha = "local"

agents.deploy(
    model_name=model_name,
    model_version=int(model_version),
    endpoint_name=endpoint_name,
    usage_policy_id=cfg.usage_policy_id,
    scale_to_zero=True,
    workload_size="Small",
    deploy_feedback_model=False,
    environment_vars={
        "GIT_SHA": git_sha,
        "MODEL_VERSION": model_version,
        "MODEL_SERVING_ENDPOINT_NAME": endpoint_name,
        "MLFLOW_EXPERIMENT_ID": experiment.experiment_id,
        "LAKEBASE_SP_CLIENT_ID": f"{{{secrets/{secret_scope}/client-id}}}",
        "LAKEBASE_SP_CLIENT_SECRET": f"{{{secrets/{secret_scope}/client-secret}}}",
        "LAKEBASE_SP_HOST": workspace.config.host,
    },
)

```

You may notice that we've provided a Databricks client ID and client secret associated with a service principal to query a table in the Lakebase instance. For this setup to work, several prerequisites must be in place: the service principal must exist, a secret scope must be configured, all required credentials must be securely stored within that scope, and the service principal must be granted the necessary permissions.

Let's walk through how to set this up step by step with the *chapter9_1.create_spn.py* notebook.

Setting up service principal authentication

Before we can proceed, we need to create a Databricks secret scope named `admin`. This scope will store the credentials of a service principal with account admin permissions, which are required to authenticate and generate a token to call to the Databricks Account API:

```

from databricks.sdk import WorkspaceClient
from requests.auth import HTTPBasicAuth

w = WorkspaceClient()

client_id = "your client id"
client_secret = "your client secret"
account_id = "your account id"

```

```

w.secrets.create_scope(scope="admin")
w.secrets.put_secret(scope="admin", key="client_id", string_value=client_id)
w.secrets.put_secret(scope="admin", key="client_secret", string_value=client_secret)
w.secrets.put_secret(scope="admin", key="account_id", string_value=account_id)

```

Let's generate an account-level token:

```

import requests
from requests.auth import HTTPBasicAuth

admin_client_id = dbutils.secrets.get("admin", "client_id")
admin_client_secret = dbutils.secrets.get("admin", "client_secret")
account_id = dbutils.secrets.get("admin", "account_id")

account_host = "https://accounts.cloud.databricks.com"

token = requests.post(
    f"{account_host}/oidc/accounts/{account_id}/v1/token",
    auth=HTTPBasicAuth(admin_client_id, admin_client_secret),
    data={"grant_type": "client_credentials", "scope": "all-apis"}
).json()["access_token"]

```

We can now create an OAuth secret for the service principal—this is the credential that the agent will use to authenticate. When creating the secret, you can optionally specify its lifetime in seconds. If you do not provide this parameter, the secret will default to a lifetime of 730 days. In our case, we haven't specified a lifetime, but it's useful to know that this option is available for stricter security controls:

```

sp = w.service_principals.create(display_name="lakebase-sp-arxiv")
secret_resp = requests.post(
    f"{account_host}/api/2.0/accounts/{account_id}/servicePrincipals/{sp.id}/credentials/secrets",
    headers={"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
)
secret_resp.raise_for_status()
client_id = sp.application_id
client_secret = secret_resp.json()["secret"]

```

We can now store the secrets in the secret scope:

```

scope_name = "arxiv-agent-scope"
try:
    w.secrets.create_scope(scope=scope_name)

```

```

except Exception:
    pass # scope already exists
w.secrets.put_secret(scope=scope_name, key="client_id", string_value=client_id)
w.secrets.put_secret(scope=scope_name, key="client_secret", string_value=client_

```

We must also add the service principal to the Lakebase project:

```

from databricks.sdk.service.postgres import (
    PostgresAPI, Role, RoleAuthMethod, RoleIdentityType, RoleRoleSpec,
)
import psycopg

project_id = "arxiv-agent-lakebase"
w = WorkspaceClient()
pg_api = PostgresAPI(w.api_client)

project = pg_api.get_project(name=f"projects/{project_id}")
default_branch = next(iter(pg_api.list_branches(parent=project.name)))
branch_parent = default_branch.name

pg_api.create_role(
    parent=branch_parent,
    role=Role(
        spec=RoleRoleSpec(
            identity_type=RoleIdentityType.SERVICE_PRINCIPAL,
            auth_method=RoleAuthMethod.LAKEBASE_OAUTH_V1,
            postgres_role=client_id,
        )
    ),
    role_id="arxiv-agent-spn",
).wait()

```

Finally, we need to assign Postgres permissions to the service principal using Postgres connection:

```

endpoint = next(iter(pg_api.list_endpoints(parent=branch_parent)))
host = endpoint.status.hosts.host
pg_credential = pg_api.generate_database_credential(endpoint=endpoint.name)

user = w.current_user.me()
username = urllib.parse.quote_plus(user.user_name)

conn_string = (
    f"postgresql://{username}:{pg_credential.token}@{host}:5432/"
    "databricks_postgres?sslmode=require"
)

with psycopg.connect(conn_string) as conn:

```

```

conn.execute(f"""
    GRANT USAGE ON SCHEMA public TO "{client_id}";
""")
conn.execute(f"""
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE session_messages TO "{client_id}";
""")
conn.execute(f"""
    GRANT USAGE, SELECT ON SEQUENCE session_messages_id_seq TO "{client_id}";
""")

```

Now that we've figured out the authentication, we can invoke the serving endpoint. Although we called a Model Serving endpoint in [Chapter 4](#), we cannot invoke this endpoint in the same way. Let's take a closer look at how it works. Note that in production setup, we'll also need to create a service principal with permissions to query the endpoint, just as we explored in [Chapter 4](#).

Authentication considerations

When using service principal credentials, the reasoning about access control is straightforward: the permissions of the service principal are consistently applied to all downstream resources. However, things become more nuanced when working with resources defined via `mlflow.models.resources` during model (or agent) logging.

MLflow resources support an *on-behalf-of-user* authentication mode. For example, a Databricks serving endpoint can be configured as follows:

```

DatabricksServingEndpoint(
    endpoint_name=cfg.llm_endpoint,
    on_behalf_of_user=True)

```

In earlier examples, we did not enable this option. As a result, the permissions of the entity that logged the model (e.g., a service principal, a specific user) are inherited and used for all subsequent interactions with the resource.

Enabling on-behalf-of-user mode changes this behavior significantly. Instead of relying on the model owner's credentials, the system executes requests by using the identity of the *calling user*. This provides us with several important advantages:

Fine-grained access control

Each user can only access data and resources they are explicitly permitted to use.

Improved security posture

It prevents over-permissioned service principals from acting as a shared identity.

Better auditability

We can trace action back to individual users, rather than a single system account.

Multi-user application support

This is especially important for shared apps (for example, internal tools) in which different users may have different entitlements.

In practice, this mode is particularly useful in enterprise environments where data governance and access policies are strict. It ensures that your LLM application respects existing permission boundaries, instead of bypassing them through a centralized identity.

In contrast, relying on the model owner's identity (which is the default behavior) can be simpler to set up, but it may unintentionally grant broader access than you've intended, especially if you use a highly privileged service principal.

Calling the Model Serving endpoint

Since we've logged the agent by using

`mlflow.pyfunc.ResponsesAgent` and it's compatible with the OpenAI Responses API, we can use the OpenAI client to call the endpoint. In the *chapter9_3.call_agent.py* notebook, we can generate our own session and request IDs and pass them as custom inputs when making the request. These identifiers will also be recorded in MLflow traces:

```
import random
from datetime import datetime
from databricks.sdk import WorkspaceClient
from openai import OpenAI

workspace = WorkspaceClient()
host = workspace.config.host
token = workspace.tokens.create(lifetime_seconds=2000).token_value
```

```

endpoint_name = "arxiv-agent-endpoint"

client = OpenAI(
    api_key=token,
    base_url=f"{host}/serving-endpoints",
)

timestamp = datetime.now().strftime("%Y%m%d-%H%M%S")
session_id = f"s-{timestamp}-{random.randint(100000, 999999)}"
request_id = f"req-{timestamp}-{random.randint(100000, 999999)}"

response = client.responses.create(
    model=endpoint_name,
    input=[
        {"role": "user", "content": "What are recent papers about LLMs and reason."},
    ],
    extra_body={"custom_inputs": {
        "session_id": session_id,
        "request_id": request_id,
    }}
)

```

So far, we’ve walked through a series of notebooks that cover data processing and agent evaluation, and we’ve moved on to logging, registering, and deploying the agent.

However, as we established in the MLOps-focused chapters, a production system consists of much more than just the agent itself. For the system to be reliable and scalable, multiple components must work together, supported by dedicated pipelines that manage the entire lifecycle. Let’s now look at how to move beyond individual notebooks and deploy the system as a whole.

Deployment Jobs

Earlier in this chapter, we discussed several core processes, and we’ll now formalize those processes into concrete pipelines using Lakeflow jobs and show you how to deploy the required resources using DABs. As we established in [Chapter 5](#), DABs provide the robust framework we need to manage and version our ML infrastructure as code.

In this chapter, we’ll cover the following Lakeflow jobs:

The data preprocessing pipeline

This pipeline runs on a schedule to process newly ingested papers and update both the Delta table and the vector search index.

The deployment pipeline

This pipeline runs on demand and is responsible for evaluating, logging, registering, and deploying the agent. In AI use cases, there is typically no continuous retraining pipeline. Instead, an agent is logged and registered only when the agent itself changes, such as when prompts, orchestration logic, tools, or LLM calling parameters are updated.

The monitoring pipeline

This pipeline fetches MLflow traces, runs evaluation, and updates the table that powers the monitoring dashboard.

Beyond the pipelines, we must also deploy the monitoring dashboard itself, and we will cover that in the next section on monitoring (together with the monitoring pipeline). While we can also use DABs to deploy additional resources like foundation model APIs (external models or provisioned throughput models) and a user-facing Databricks app, they fall outside the scope of this chapter.

Let's now examine each of these Lakeflow jobs one by one.

Data preprocessing job

So far, we have defined the end-to-end process for discovering new AI papers in the arXiv database, storing the raw documents in a volume, extracting and persisting metadata in a Delta table, and parsing and chunking the papers to keep the vector search index up to date. The next step is to deploy a pipeline that does it on a *daily schedule*.

Although Lakeflow Jobs are not new to us, there are a few differences in how we use them in this chapter that are worth noting. In the MLOps chapters, we relied on classic compute for our jobs. At the time of writing, this was the better choice for certain ML workloads, as some features, such as hyperparameter tuning with Ray Tune, were not yet available in serverless compute. Additionally, ML workflows often involve large datasets that must be loaded into memory, and serverless environments may not always provide sufficient resources for those scenarios.

That said, Databricks continues to invest heavily in serverless compute, and new capabilities are being added regularly, so it's an increasingly strong option for a wider range of workloads. For LLM applications, serverless compute is the natural choice. Some features, such as the express deployments for Model Serving endpoints we used earlier, are not

available on classic compute at all. That is why we use serverless throughout this chapter.

Another difference is that instead of defining a `spark_python_task`, we are using a notebook task. We do this partly for demonstration purposes, to showcase an alternative approach. In general, notebook tasks can be easier to manage because file paths are resolved relative to the notebook's location. This means you don't have to worry about where dependent files reside, and execution remains consistent whether the notebook is run from VS Code or as part of a job.

The data preprocessing job is defined under *resources/process_data.yml*. Here, we set the schedule, specify serverless environment version 4, and define one task that runs the *process_data.py* notebook under the *deployment_notebooks* folder:

```
resources:
  jobs:
    data-pipeline:
      name: arxiv-data-pipeline
      schedule:
        quartz_cron_expression: "0 0 6 * * ?"
        timezone_id: "Europe/Amsterdam"
        pause_status: "${var.schedule_pause_status}"
        budget_policy_id: "${var.usage_policy_id}"
      environments:
        - environment_key: default
          spec:
            environment_version: "4"
            dependencies:
              - ../dist/*.whl

      tasks:
        - task_key: preprocessing
          environment_key: default
          notebook_task:
            notebook_path: deployment_notebooks/process_data.py
            base_parameters:
              env: "${bundle.target}"
```

The *process_data.py* notebook contains the same code we walked through earlier, but it's packaged into reusable modules to keep the code clean and easy to test. The code we just reviewed is also packaged into reusable modules. The `DataProcessor` class is defined in the `data_processor` module, while the `VectorSearchManager` class lives in the `vector_search` module:

```

import yaml
from loguru import logger
from pyspark.sql import SparkSession

from arxiv_curator.config import ProjectConfig
from arxiv_curator.data_processor import DataProcessor
from arxiv_curator.utils.common import get_widget
from arxiv_curator.vector_search import VectorSearchManager

env = get_widget("env", "dev")

project_config = ProjectConfig.from_yaml(
    "../..//project_config.yml", env=env)
logger.info("Configuration loaded:")
logger.info(yaml.dump(project_config, default_flow_style=False))

spark = SparkSession.builder.getOrCreate()

# Process data
data_processor = DataProcessor(
    config=project_config,
    spark=spark)
records = data_processor.download_and_store_papers()

if records is None:
    logger.info("No new papers to process. Exiting.")
    pass
else:
    data_processor.parse_pdfs_with_ai()
    logger.info("Parsed documents.")

    data_processor.process_chunks()
    logger.info("Processing complete!")

# Sync vector search index
vector_search_manager = VectorSearchManager(config=project_config)
vector_search_manager.sync_index()

```

Notice that we use the `get_widget` function defined in the `utils.common` module. This helper is designed to retrieve parameters passed to a task and fall back to default values when the code is executed locally, rather than as part of a job:

```

from databricks.sdk.runtime import dbutils

def get_widget(name: str, default: str | None = None) -> str | None:

```

```

"""Get a Databricks widget value with a fallback default.

:param name: Widget name.
:param default: Default value if widget is not set.
:return: Widget value or default.
"""

try:
    return dbutils.widgets.get(name)
except KeyError:
    return default

```

We use this function across all our notebooks to ensure consistent parameter handling. Now, let's examine another set of notebooks and the corresponding deployment job that is designed to execute them in sequence.

Agent deployment job

The agent deployment workflow consists of two stages: first, logging and registering the agent, and then, deploying it. Once again, we use a serverless compute environment and define the workflow using notebook tasks.

An interesting design choice we make here is to implement each stage as a separate job, orchestrated by a parent workflow. This allows us to run each step independently whenever we need to, which is especially useful when we want to debug, test, or re-run only a specific part of the pipeline without executing everything end-to-end.

The trade-off is that, at the time of writing, task values cannot be passed directly between jobs. This means we must handle any data sharing between stages explicitly. In our case, we use the `latest-model` stage as a handoff mechanism.

In development environments, this approach can introduce a potential race condition if multiple runs overlap. However, this is unlikely to be an issue in higher environments, where deployments are more controlled and sequential:

```

resources:
  jobs:
    log_register_agent_job:
      name: arxiv-agent-log-register
      budget_policy_id: "${var.usage_policy_id}"
      environments:
        - environment_key: default
          spec:

```

```

        environment_version: "4"
        dependencies:
            - ../dist/*.whl

tasks:
  - task_key: log_register_agent
    environment_key: default
    notebook_task:
        notebook_path: deployment_notebooks/log_register_agent.py
        base_parameters:
            env: "${bundle.target}"
            git_sha: "${var.git_sha}"
            run_id: "${job.run_id}"

deploy_agent_job:
  name: arxiv-agent-deploy
  budget_policy_id: "${var.usage_policy_id}"
  environments:
    - environment_key: default
      spec:
        environment_version: "4"
        dependencies:
            - ../dist/*.whl

tasks:
  - task_key: deploy_agent
    environment_key: default
    notebook_task:
        notebook_path: deployment_notebooks/deploy_agent.py
        base_parameters:
            env: "${bundle.target}"
            git_sha: "${var.git_sha}"

register_deploy_agent:
  name: arxiv-agent-register-deploy-pipeline
  budget_policy_id: "${var.usage_policy_id}"

tasks:
  - task_key: run_log_register_agent
    run_job_task:
        job_id: "${resources.jobs.log_register_agent_job.id}"

  - task_key: run_deploy_agent
    depends_on:
        - task_key: run_log_register_agent
    run_job_task:
        job_id: "${resources.jobs.deploy_agent_job.id}"

```

Let's walk through the *log_register_agent.py* notebook. Parameters such as `env`, `git_sha`, and `run_id` are passed to the task. The notebook first

runs the evaluation on the evaluation dataset, then logs the model together with its evaluation metrics, and finally registers it:

```
# Databricks notebook source
from databricks.sdk.runtime import dbutils

from arxiv_curator.agent import log_register_agent
from arxiv_curator.config import ProjectConfig
from arxiv_curator.evaluation import evaluate_agent
from arxiv_curator.utils.common import get_widget

env = get_widget("env", "dev")
git_sha = get_widget("git_sha", "local")
run_id = get_widget("run_id", "local")

cfg = ProjectConfig.from_yaml(
    config_path="../../project_config.yml",
    env=env)

mlflow.set_experiment(cfg.experiment_path)
model_name = f"{cfg.catalog}.{cfg.schema}.arxiv_agent"

# COMMAND -----
# Run evaluation
results = evaluate_agent(
    cfg,
    eval_inputs_path="../../eval_inputs.txt")

# COMMAND -----
# Log and register model
registered_model = log_register_agent(
    cfg=cfg,
    git_sha=git_sha,
    run_id=run_id,
    agent_code_path="../../arxiv_agent.py",
    model_name=model_name,
    evaluation_metrics=results.metrics,
)
```

The *deploy_agent.py* notebook follows the same logic as the one we discussed earlier, so we won't go through it again in detail. Again, the model itself is resolved using the `latest-model` alias, which acts as the hand-off point between the registration and deployment stages. The main difference is that the `env` and `git_sha` values are passed dynamically via parameters:

```
from arxiv_curator.utils.common import get_widget
```

```
git_sha = get_widget("git_sha", "local")
env = get_widget("env", "dev")
```

Now, we can run the **databricks bundle deploy** command from VS code, and the pipelines will get deployed to the development environment. The *databricks.yml* file will look exactly like we defined it earlier in [Chapter 5](#).

We can then trigger the pipeline run by executing the **databricks bundle run <job name>** command and get our agent with all necessary resources running. However, this raises an important operational question: how does the promotion across environments work? Let's discuss deployment considerations.

Deployment Considerations

In [Chapter 5](#), we discussed three environments—`dev`, `acc`, and `prd`—and how code promotion works across them. We followed a GitHub flow: developers work in feature branches, create pull requests to merge into the main branch, and require approvals before merging. The CI pipeline runs tests and pre-commit checks, and only after it succeeds can changes be merged. Once changes are merged, resources are deployed to the `acc` environment using **databricks bundle deploy**, and after approval, they're deployed to `prd`.

In the context of traditional ML, we assumed that code changes would trigger model retraining in each environment, with periodic retraining scheduled in production. We also discussed scenarios in which trained models could be shared across environments.

Agents, however, change this dynamic. The debate between *model promotion* and *code promotion* is central in MLOps, but it is far less relevant in LLMOps. In traditional ML systems, retraining models in every environment is often costly and time-consuming. A common pattern is to train once and promote the resulting model artifact across environments.

For AI applications, this pattern usually does not apply. Most companies do not fine-tune foundation models, and even when they do, the usage pattern is the same as for any other foundation model API: the model is served behind an endpoint and called the same way. The model is not the artifact you promote. Instead, AI systems are composed of artifacts such as tool definitions, LLM API calls, prompts, orchestration logic, vector search configuration, and (potentially) a database for agent memory.

Prompts and orchestration logic are generally environment agnostic, while other components, such as AI Search indexes and external connections, may be environment specific. As a result, we must execute and deploy the system in each environment. There is typically no standalone “model artifact” to promote. In this sense, AI applications are closer to traditional software systems than to classic ML workflows.

The agent deployment pipeline therefore runs on demand and must be triggered in both `acc` and `prd`. In practice, the CD pipeline should always include a `databricks bundle run` command to execute the deployment job in the target environment.

As you may have noticed, this LLMOps chapter follows the same pattern as previous chapters devoted to MLOps, and there is one aspect left: LLM system monitoring.

LLM System Monitoring

So far, we’ve focused on evaluation. Monitoring is its natural continuation. While [Chapter 6](#) covered monitoring in the context of traditional ML systems, LLM-based systems introduce additional considerations that require us to use a slightly different approach.

In traditional ML use cases, evaluation is typically inexpensive. As long as we have ground truth labels, we can join predictions with the ground truth and compute metrics across the full dataset. By default, this means calculating metrics over all records.

Evaluation of an AI agent, however, can be significantly more expensive. Code-based metrics are relatively cheap to compute, but LLM-as-a-Judge and other LLM-based scorers quickly add costs in terms of both processing time and API usage. If your agent processes thousands of requests per day, running full LLM-based evaluation on every request becomes impractical.

A more scalable approach is a *tiered evaluation strategy*: you apply inexpensive, code-based checks to all requests, and you run more expensive LLM-based evaluations on a representative sample.

What to Monitor in LLM Systems

For every inference (and its corresponding trace), we should monitor the following:

Latency and token counts

These are the core operational metrics.

Tool usage

This is which tools were invoked and how often.

Format validation

This is whether the output matches the expected structure.

Guardrail violations

This is whether safety filters were triggered.

Error rates

This is whether and how often the agent failed to produce a valid response.

Simple heuristics

These include length checks, regex validation, and keyword detection.

MLflow code-based scorers can be used for format validation and heuristic checks, while the remaining metrics can be computed directly from inference and trace tables. These form *Tier 1 metrics*, which are inexpensive to compute and suitable for near real-time visibility into agent behavior.

Tier 2 metrics include LLM-based scorers such as relevance, correctness, and tone analysis. Running them on every request is expensive, so they are applied to a representative sample rather than every request. Databricks recommends a sample rate of 5–20% for LLM judges, which provides enough signal to catch quality regressions without the cost of scoring all traffic.³

When designing your sampling strategy, you should consider whether simple random sampling is sufficient or whether stratification is needed. Random sampling works well for general monitoring, but you may want to oversample specific categories like low-confidence responses, interactions from certain user segments, and requests that trigger edge-case guardrails.

This practical architecture separates trace collection from evaluation:

1. MLflow traces are synchronized to Delta tables. This is enabled by default via the agent framework.
2. A scheduled processing job reads non-evaluated traces, computes Tier 1 metrics for all records, selects a sample for Tier 2 evaluation, and runs the additional scoring.
3. The evaluation results are logged to MLflow and become available in the Delta table.

Both tiers can run within a single job or in separate jobs. Running them separately offers greater flexibility: code-based evaluation might run every fifteen minutes, while LLM-based evaluation runs hourly. In our example, we will have a single job.

Let's now proceed with implementing this logic.

Monitoring Implementation

Before going into depth, we should note that MLflow traces are automatically synchronized to Delta tables via the Agents framework (through the `agents.deploy()` function). [Table 9-2](#) outlines the structure of the Delta table.

Table 9-2. Structure of the synchronized Delta table with traces

Column	Type	Description
trace_id	string	Unique identifier for the trace
client_request_id	string	Request ID passed from the client
request_time	timestamp	Time when the request was received
state	string	Execution state (e.g., success, error)
execution_duration_ms	bigint	Total execution time in milliseconds
request	string	Full request payload
response	string	Full response payload
request_preview	string	Truncated preview of the request
response_preview	string	Truncated preview of the response
trace_metadata	map<string, string>	Metadata associated with the trace (e.g., session ID)
tags	map<string, string>	Tags such as git_sha, model version, and endpoint name
trace_location	struct	Location of the trace (MLflow experiment or inference table)
assessments	array<struct>	Evaluation results, including feedback, expectations, and metadata
spans	array<struct>	Detailed execution spans with timing, hierarchy, and attributes

For monitoring purposes, we are interested in the trace id, total execution duration (latency), number of tool calls, number of LLM calls, and token usage. We'll use the trace ID to run the evaluation we defined earlier.

As with the other jobs we discussed earlier, the deployment follows a familiar pattern: a notebook contains the logic, and a Lakeflow job executes it via a notebook task.

Deployment notebook

The *production_monitoring.py* notebook is available in the *resources/deployment_notebooks* folder. For readability, we'll omit the import statements from the snippets below.

The `traces_table` is located in the same catalog and schema where the model was registered, with a name that follows the `trace_logs_{experiment_id}` convention. This table contains the raw MLflow traces that serve as input into the evaluation pipeline:

```
catalog = cfg.catalog
schema = cfg.schema

experiment = MlflowClient().get_experiment_by_name(cfg.experiment_path)
traces_table = f"{catalog}.{schema}.trace_logs_{experiment.experiment_id}"
```

We first retrieve all traces that have not yet been evaluated and load them into a PySpark DataFrame. Since the `mlflow.genai.evaluate()` function operates on Pandas DataFrames that follow certain formats, we then convert the data accordingly:

```
new_traces_df = spark.sql(f"""
    SELECT * FROM (
        SELECT
            t.trace_id,
            t.request_preview,
            element_at(
                filter(
                    from_json(
                        get_json_object(t.response, '$.output'),
                        'ARRAY<STRUCT<type:STRING, content:ARRAY<STRUCT<text:STRING>>'
                    ),
                    x -> x.type = 'message'
                ),
                1
            ).content[0].text AS response_text
        FROM {traces_table} t
        WHERE tags['model_serving_endpoint_name']
            = 'arxiv-agent-endpoint'
            AND (t.assessments IS NULL OR size(t.assessments) = 0)
    ) WHERE response_text IS NOT NULL
""")

traces_pdf = new_traces_df.toPandas()

eval_pdf = pd.DataFrame({
```

```

"trace_id": traces_pdf["trace_id"],
"inputs": traces_pdf["request_preview"].apply(
    lambda x: {"query": x}
),
"outputs": traces_pdf["response_text"],
})

```

For all traces in the resulting DataFrame, we run the *Tier 1 evaluation* (in this case, a word count check) and log the feedback back to MLflow:

```

wc_result = mlflow.genai.evaluate(
    data=eval_pdf[["inputs", "outputs"]],
    scorers=[word_count_check],
)

for trace_id, assessments in zip(
    eval_pdf["trace_id"],
    wc_result.result_df["assessments"],
    strict=True,
):
    val = assessments[0]["feedback"]["value"]
    mlflow.log_feedback(
        trace_id=trace_id,
        name="word_count_check",
        value=val,
    )

```

Next, we take a 10% sample of the traces and perform a *Tier 2 evaluation*, which includes more advanced, LLM-based checks such as for tone (politeness) and content quality (the presence of a hook in the post):

```

sample_size = max(1, int(len(eval_pdf) * 0.1))
sampled_pdf = eval_pdf.sample(n=sample_size)
logger.info(
    f"Sampled {len(sampled_pdf)} traces for LLM-based evaluation"
)

llm_result = mlflow.genai.evaluate(
    data=sampled_pdf[["inputs", "outputs"]],
    scorers=[polite_tone_guideline, hook_in_post_guideline],
)

for trace_id, assessments in zip(
    sampled_pdf["trace_id"],
    llm_result.result_df["assessments"],
    strict=True,
):

```

```

for a in assessments:
    name = a["assessment_name"]
    val = a["feedback"]["value"]
    mlflow.log_feedback(
        trace_id=trace_id,
        name=name,
        value=val,
    )

```

Lakeflow job

Finally, we execute the notebook as part of a Lakeflow job, which is defined in *production_monitoring.yml*. The environment configuration follows the same pattern as the other Lakeflow jobs we covered earlier.

In this example, the job is scheduled to run daily at 7:00, though we can adjust the frequency to match the needs of our use case:

```

resources:
  jobs:
    update-and-evaluate-traces:
      name: arxiv-update-and-evaluate-traces
      schedule:
        quartz_cron_expression: "0 0 7 * * ?"
        timezone_id: "Europe/Amsterdam"
        pause_status: "${var.schedule_pause_status}"
        budget_policy_id: "${var.usage_policy_id}"
      environments:
        - environment_key: default
          spec:
            environment_version: "4"
            dependencies:
              - ../dist/*.whl

      tasks:
        - task_key: update_and_evaluate
          environment_key: default
          notebook_task:
            notebook_path: deployment_notebooks/production_monitoring.py
            base_parameters:
              env: "${bundle.target}"

```

Once the traces table {catalog}.

{schema}.trace_logs_{experiment_id} refreshes, the logged feedback becomes available alongside the original traces, which makes it easy for us to analyze evaluation results over time.

In practice, we create a view on top of the trace logs table and use it as the data source for a dashboard. We won't go into the details of constructing this view here, as it involves a fairly large query. Refer to the book's source code for the full implementation.

Monitoring dashboard

Databricks dashboard is great for monitoring AI applications. This setup keeps everything within a single governed environment. Since dashboards query Unity Catalog directly, access control remains consistent and there is no need for us to replicate permissions or manage separate BI tooling. This significantly reduces operational overhead and avoids common issues with access drift.

[Genie Code](#) simplifies dashboard creation by acting as an AI-powered assistant for authoring artificial intelligence and business intelligence (AI/BI) dashboards. It's available through Agent mode, and it can generate visualizations, configure layouts, and even build complete dashboards from a single prompt. That significantly reduces the barriers to getting started.

Using this approach, I created a dashboard to visualize key metrics, such as tokens used per minute, the number of LLM and tool calls, and the percentage of passed assessments over time (see [Figure 9-2](#)).



Figure 9-2. Monitoring dashboard

Once you create a dashboard, you can export it as an `.lvdash.json` file that you can use to deploy using DABs (see the `dashboard/agent_monitoring_dashboard.yml` file under the `resources` folder):⁴

```
resources:
  sql_warehouses:
    arxiv_monitoring_warehouse:
      name: arxiv_monitoring_warehouse
      cluster_size: 2X-Small
      enable_serverless_compute: true
      max_num_clusters: 1
      auto_stop_mins: 5
      warehouse_type: PRO
      tags:
        custom_tags:
          - key: "project"
            value: "arxiv_curator"
  dashboards:
    agent-monitoring-dashboard:
      display_name: "Agent Monitoring Dashboard"
      file_path: agent_monitoring_dashboard.lvdash.json
      warehouse_id: ${resources.sql_warehouses.arxiv_monitoring_warehouse.id}
```

Monitoring considerations

We’ve walked through a simple setup for monitoring AI agents, but there are a few important considerations to keep in mind.

A dashboard on its own is not sufficient: you also need to define alerts based on the underlying traces table. For example, you may want to trigger alerts when metrics such as average latency, tool usage, or token consumption exceed predefined thresholds. We covered how to set up such alerts in [Chapter 6](#), so we won’t repeat the implementation here.

In our evaluation pipeline, we relied on guideline-based and code-based metrics rather than LLM judges. However, if you use LLM-based judges, logging feedback to MLflow will provide additional benefits. It will allow you to identify traces evaluated by an LLM, route a subset for human review, and compare the results.

This enables alignment between human and LLM judgments over time—something that you can implement as a separate Lakeflow job if you need to. As we discussed earlier in the MLflow GenAI evaluation framework, this has a direct impact on both evaluation quality and ongoing monitoring.

Cost Monitoring

The metrics we have collected so far describe how the agent behaves. They say nothing about what it costs to run. We put off cost monitoring until now on purpose: it only becomes concrete once there are running workloads to attribute spend to. Across this book, we have built two projects—the arXiv curator and the hotel booking price predictor—and applied a usage policy to each.

Every unit of serverless consumption that either project generates is now tagged against its policy, so the billing records carry the attribution we need to separate one project's spend from the other's. That attribution is what makes the rest of this section possible. We can break down the costs by project and by workload, and we can monitor them over time.

This section covers only what you need to monitor the costs of your projects and resources. For a broader view of FinOps on Databricks, see [“The FinOps Guide to Databricks” blog post](#) by Casper Lubbers. This section of the book was largely inspired by it.

Databricks costs consist of two components: what Databricks charges you for its software and what your cloud provider charges you for the infrastructure underneath, such as virtual machines (VMs), storage, and networking.

This split applies to classic compute, in which Databricks runs the VMs in your own cloud account. With serverless, the compute runs in Databricks' account instead, so it counts as a Databricks cost rather than a cloud provider one. Almost all of the code in this book runs on serverless, so we focus throughout on the Databricks side, which is also where the complexity in cost attribution lives.

How Databricks billing works

Databricks tracks usage in *Databricks Units* (DBUs). A DBU is a measure of processing capability per hour, but its price is not fixed: it varies with the service, the workspace tier (Standard versus Premium), and the cloud region.

Most Databricks services bill in DBUs, and for most organizations, DBUs dominate the bill. A few other units cover the rest, and these include the Databricks Storage Unit (DSU) for managed storage and the gigabyte (GB) for data transfer.⁵

The non-DBU units are easy to overlook. Storage and networking in particular can become meaningful at scale, especially in architectures that rely on cross-region replication or Delta Sharing, where DSU and GB charges add up.

Every stock-keeping unit (SKU) has a list price that is visible in `system.billing.list_prices`. The table exposes the `default` and the `effective_list` price, which reflects promotions. Neither one is your negotiated rate. Applying these prices to your usage gives you an estimate, not the actual amount you are billed.⁶

Databricks services and associated costs

Every `system.billing.usage` record includes a `billing_origin_product` and a `sku_name`:

billing_origin_product

This is the service that generated the cost, such as Databricks AI Search (previously, Vector Search) or AI Functions.

sku_name

This is how that usage is priced, and it maps to a row in `system.billing.list_prices`.

The mapping between them is not one-to-one. As [Table 9-3](#) shows, one service can bill under several SKUs (even in different usage units) and the same SKU can serve several services.

Table 9-3. Mapping of `billing_origin_product` and `sku_name`

<code>billing_origin_product</code>	<code>sku_name</code>	<code>usage_unit</code>
AI_FUNCTIONS	PREMIUM_SERVERLESS_REAL_TIME_INFERENCE	DBU
AI_GATEWAY	PREMIUM_SERVERLESS_REAL_TIME_INFERENCE	DBU
LAKEBASE	PREMIUM_DATABASE_SERVERLESS_COMPUTE	DBU
LAKEBASE	PREMIUM_DATABRICKS_STORAGE	DSU
MODEL_SERVING	PREMIUM_ANTHROPIC_MODEL_SERVING	DBU
MODEL_SERVING	PREMIUM_SERVERLESS_REAL_TIME_INFERENCE	DBU
VECTOR_SEARCH	PREMIUM_JOBS_SERVERLESS_COMPUTE	DBU
VECTOR_SEARCH	PREMIUM_SERVERLESS_REAL_TIME_INFERENCE	DBU
VECTOR_SEARCH	PREMIUM_SERVERLESS_REAL_TIME_INFERENCE	COUNT

[Table 9-3](#) was produced by the following query, which strips the region suffix from `sku_name` with `regexp_replace`. Each SKU name ends in its region, such as `_EU_WEST`, so that SKU appears once per region:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

spark.sql("""
    SELECT DISTINCT
        billing_origin_product,
        regexp_replace(
            sku_name, '_(US_EAST|EU_WEST|US_EAST_2|US_WEST|US_WEST_2)$', ''
        ) AS sku_name,
        usage_unit
    FROM system.billing.usage
    WHERE billing_origin_product IN (
        'VECTOR_SEARCH', 'AI_FUNCTIONS', 'MODEL_SERVING',
        'FOUNDATION_MODEL_APIS', 'AI_GATEWAY', 'LAKEBASE'
    )

```

```
ORDER BY billing_origin_product, sku_name
""").display()
```

You should group usage by both `billing_origin_product` and `sku_name`, rather than by either one alone. Together, they show which service drives the cost and how that usage is priced, which neither field reveals on its own.

What matters most here is the cost per project, and neither field captures it. That attribution comes from tagging.

Tag-based cost attribution

Cost attribution in Databricks comes down to one thing: getting the right tags onto the right resources so they appear in `system.billing.usage.custom_tags`.

For serverless resources, you should attribute cost with a usage policy wherever possible. A *usage policy* is an account-level object that holds a set of tags.⁷ You can grant it to users and groups, and the resources they create carry the policy's tags on every serverless usage record.

A usage policy covers serverless notebooks, jobs, pipelines, Model Serving endpoints, Databricks AI Search endpoints, and other serverless resources. You cannot apply it to classic compute resources. Where a usage policy cannot apply, you should tag the resource directly.

On serverless, the two can collide, and the policy always wins. A usage policy is applied to every serverless resource even when none is specified, and it defaults to whichever of the user's policies comes first alphabetically. In our setup, that is `llmops_usage_policy`, which sets `project: arxiv_curator`. Because policy tags override a direct tag with the same key, a job you tag with `project: hotel_booking` is still billed as `project: arxiv_curator`.

Make sure to assign the right usage policy explicitly to every serverless resource to keep attribution correct. [Table 9-4](#) lists all the resources we used in the book and how we applied tagging to these resources.

Table 9-4. Resources and tag attribution mechanisms

Resources	Tag attribution mechanisms
Model Serving endpoint	Usage policy via SDK
Databricks Online store	Usage policy via SDK
AI Search endpoint and index	Usage policy via SDK
Lakebase project	Usage policy via SDK
Serverless jobs (pipelines, tests, and agent deployments)	Usage policy via DABs
Classic compute monitoring job	Direct tag via DABs
SQL warehouse	Direct tag via SDK/DABs

Notice that two resources cannot take a usage policy, so they keep a direct tag. The monitoring job runs on classic compute, where the serverless usage policy does not apply, so it carries an explicit project tag. SQL warehouses are the second exception: at the time of writing, serverless SQL warehouses do not support a usage policy, so the warehouses behind our alerts and monitoring dashboard are tagged directly instead.

To keep the policy ID in one place rather than scattered across resource files, we define it once as a bundle variable in *databricks.yml*. The SDK-created resources could take the same bundle variable, but during development, it is more convenient to read the ID from *project_config.yml*, which they load at runtime. That leaves two copies of the ID, one in each file, and they must stay aligned. If they drift, the bundle's jobs and the SDK-created resources end up tagged under different policies.

Tag values need the same discipline. Policy-covered resources inherit the policy's value and directly tagged resources carry their own, so the two must agree on both their key and their value to group cleanly.

Cost-monitoring dashboard

With the tags aligned, costs are grouped cleanly by project. For visualization purposes, we will reuse an existing dashboard rather than building a new one. Databricks ships a prebuilt usage dashboard that breaks spend down by the dimensions we tag, and we can deploy it from the account console or with the Databricks SDK.⁸ Building a custom dashboard is an option if you need one, but the prebuilt dashboard will cover what we need here, so we will not build our own.

The dashboard's Usage Analysis—Tag Matching tab shows usage per product for a given tag. [Figure 9-3](#) shows usage for `project=hotel_booking`, and [Figure 9-4](#) shows usage by product for `project=arxiv_curator`. These numbers do not represent real project cost: I built and reworked both projects during more than a year of writing, and the dashboard shows only the last month, so production usage will look different.

What they do show is which resources cost little when idle and which keep billing. Model Serving endpoints and Lakebase scale to zero, so they barely show any cost. Databricks AI Search (which shows up as `VECTOR_SEARCH`) does not: an endpoint keeps billing until you delete it, and there's no way to pause it, so while writing the book, I deleted and re-created it as needed.

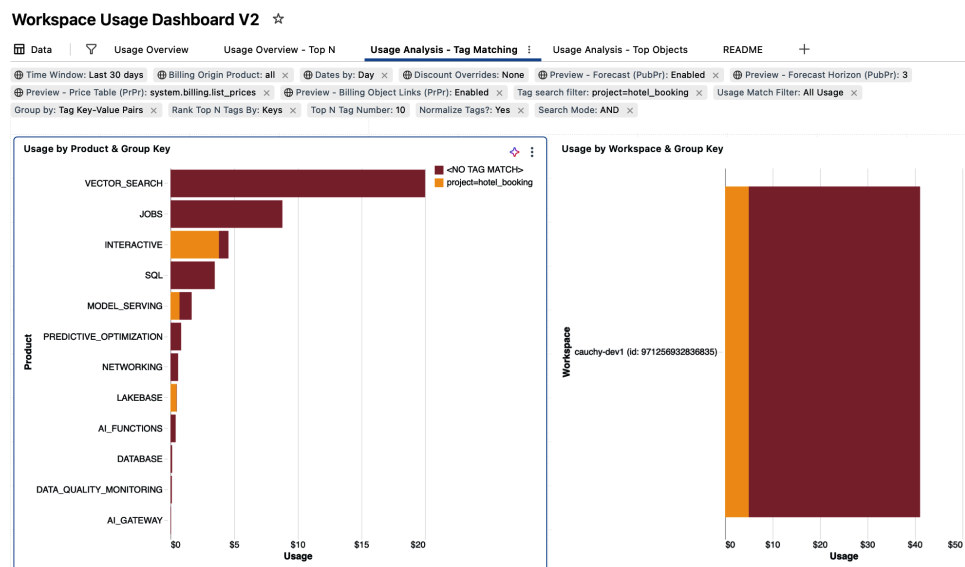


Figure 9-3. Usage Analysis—Tag Matching tab filtered for `project=hotel_booking`

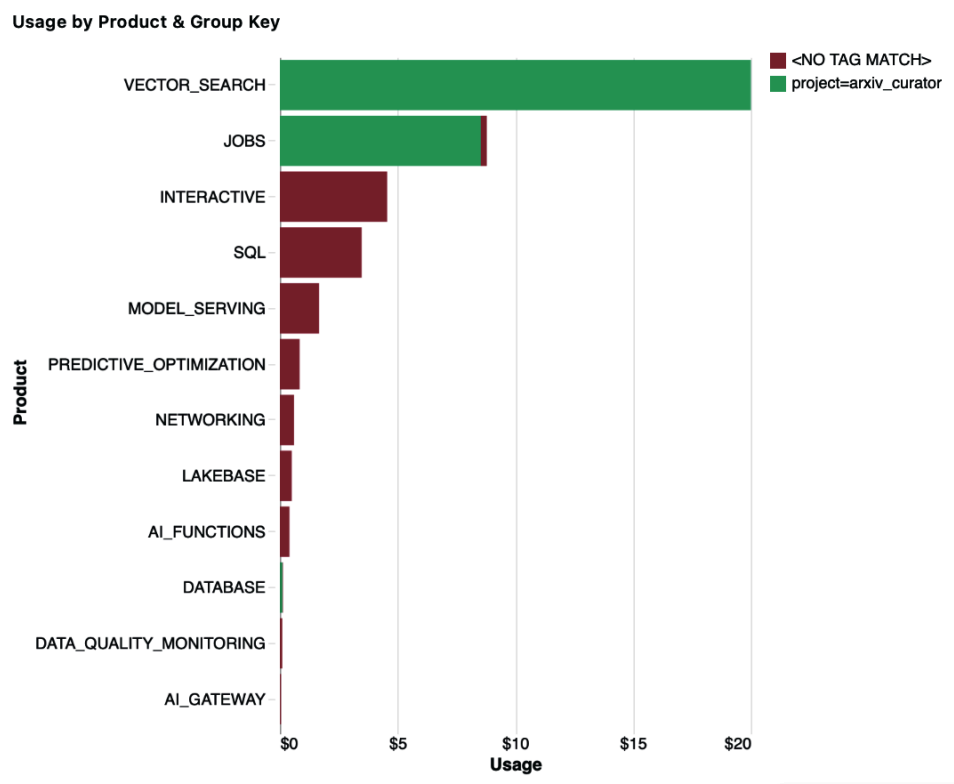


Figure 9-4. Usage Analysis—Tag Matching tab with usage per product filtered for `project=arxiv_curator`

The matching tag analysis does not capture foundation model usage. The arXiv curator queries pay-per-token foundation models, and billing alone cannot tie that usage to the project, so we attribute it through tracing instead.

Attribution is easier when an endpoint serves a single project. An external model endpoint or a dedicated serving endpoint used by only one project attributes to that project by endpoint name, and you can tag it directly since you created it.

External endpoints split the bill. When you call an external provider through a Databricks external model endpoint, Databricks bills only the gateway side while the provider bills the tokens on its own invoice. Tagging the endpoint captures the Databricks-side cost; the provider’s token charge sits outside `system.billing.usage` entirely.

The dashboard depicted in [Figure 9-5](#) shows the top endpoints by spend, but it does not include the per-project token usage from the tracing tables.⁹ Joining the two is not straightforward because each use case keeps its own tracing tables.

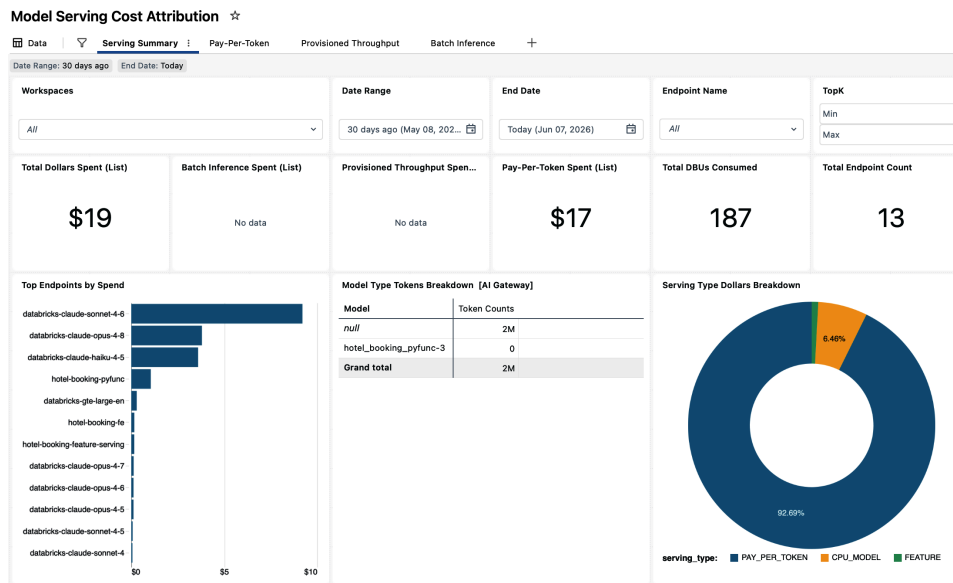


Figure 9-5. Model Serving cost attribution dashboard

Cost attribution on Databricks requires you to perform deliberate setup. You assign usage policies to serverless resources, keep tag keys and values consistent across policies and direct tags, and rely on additional sources when the billing tables are incomplete.

FinOps is a hard discipline, and this section has covered only its cost-monitoring and attribution side. This concludes our discussion of monitoring, from behavioral metrics and evaluation through to cost.

Conclusion

Congratulations on finishing the LLMOps part of the book! We’ve covered the most important aspects of bringing an LLM-powered agent into production: context engineering, tracing, evaluation, logging and registration, deployment pipelines, monitoring, and observability. While the example we built was intentionally simple, the patterns and architectural decisions we discussed are directly applicable to real-world AI systems.

It’s worth emphasizing that this chapter focused primarily on the *operational side* of LLM-powered systems. Building truly great agents—ones that are robust, intelligent, and deeply aligned with user needs—goes beyond what we’ve covered here. Topics such as advanced prompt design, fine-tuning strategies, and guardrails deserve much deeper exploration on their own.

If you’d like to go further, consulting the following resources is an excellent next step:

- [*Building LLMs for Production*](#) by Louis-François Bouchard and Louie Peters (O'Reilly). This is a hands-on guide to LLMs, prompting, RAG, and fine-tuning.
- [*LLMs in Production*](#) by Matthew Sharp and Christopher Brousseau (Manning). This book focuses on turning LLMs into real products, and it includes strong coverage of system design and deployment trade-offs.
- [*LLM Engineer's Handbook*](#) by Paul Iusztin and Maxime Labonne (Packt). This is a hands-on guide to modern LLM workflows, from experimentation to production.
- [*AI Engineering*](#) by Chip Huyen (O'Reilly). This is one of the most relevant modern books on designing and operating AI systems.
- [*Hands-On Large Language Models*](#) by Jay Alammar and Maarten Grootendorst (O'Reilly). This is a practical and intuitive introduction to how LLMs work and how to use them effectively.

The space of LLMOps is evolving rapidly. The tools, abstractions, and best practices you've seen in this chapter will continue to mature, but the core principles remain the same: reproducibility, observability, controlled deployment, and continuous evaluation.

In the next chapter, we'll take a deep dive into AI governance and explore how the practices we've applied so far help us align with emerging AI regulations.

- ¹ You can consult [the documentation on trust in and the safety of Databricks AI assistive features](#).
- ² For more details, check out [the MLflow documentation](#).
- ³ Check out [the Databricks documentation on production monitoring](#).
- ⁴ For more details, consult [the Databricks documentation](#).
- ⁵ The `usage_unit` column in `system.billing.list_prices` records the unit for every priced item, so selecting its distinct values returns the complete set that's in use for your account. Because Databricks adds billable services over time, this query stays current where a fixed list would not.
- ⁶ Your negotiated rate, which is set in a private commitment or enterprise agreement, does not appear in `system.billing.list_prices` or any other system table.
- ⁷ A *usage policy* was previously known as a *budget policy*. You can find the term *budget policy* in some documentation and API/ SDK implementations

⁸ Check out [the Databricks SDK documentation](#). You can also download the dashboard from [the Databricks Labs GitHub](#).

⁹ This [dashboard](#) was created by Databricks solution architects.

Previous chapter

< [8. MLflow for GenAI](#)

Next chapter

[10. AI Governance](#) >